

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.12.24 星期三 11-12 节

实验题目：添加新的系统调用（文件拷贝）

一、实验内容

为Linux内核（6.16.12）添加一个带参数的系统调用 `alcall`，支持内核态文件拷贝，实现用户态传递文件路径参数给内核，内核完成拷贝后返回结果，编写用户态测试程序，验证新系统调用的功能。

二、实现步骤

1、环境准备

运行Linux 6.16.12 内核并进入内核源码目录

```
LZH2311451@LZH2311451-pc:~$ uname -a
Linux LZH2311451-pc 6.16.12LZH2311451 #3 SMP PREEMPT_DYNAMIC Thu Jan 1 08:41:41
CST 2026 x86_64 x86_64 x86_64 GNU/Linux
LZH2311451@LZH2311451-pc:~$ cd /usr/src/linux
LZH2311451@LZH2311451-pc:/usr/src/linux$
```

2、添加系统调用

（1）声明系统调用原型

`vim include/linux/syscalls.h`

```
asmlinkage long sys_alcall(char *source, char *target);
#endif /* CONFIG_ARCH_HAS_SYSCALL_WRAPPER */
```

（2）实现系统调用功能

`vim kernel/sys.c`

```

SYSCALL_DEFINE2(alcall, char*, source, char*, target)
{
    int value;
    int count = 0;
    struct file* src_file = NULL;
    struct file* tgt_file = NULL;
    loff_t src_pos;
    loff_t tgt_pos;
    char buffer[128];
    char* src = kmalloc(sizeof(char)*30, GFP_ATOMIC);
    char* tgt = kmalloc(sizeof(char)*30, GFP_ATOMIC);

    value = copy_from_user(src, source, 30);
    value = copy_from_user(tgt, target, 30);

    printk("LZH2311451: copy %s to %s", src, tgt);
    src_file = filp_open(src, O_RDWR | O_APPEND | O_CREAT, 0644);
    tgt_file = filp_open(tgt, O_RDWR | O_APPEND | O_CREAT, 0644);

    if (IS_ERR(src_file)) {
        printk("failed to open file");
        return 0;
    }

    if (IS_ERR(tgt_file)) {
        printk("failed to open file");
        return 0;
    }

    src_pos = src_file->f_pos;
    tgt_pos = tgt_file->f_pos;

    while((count = kernel_read(src_file, buffer, 128, &src_pos)) > 0)
    {
        kernel_write(tgt_file, buffer, count, &tgt_pos);
    }

    filp_close(src_file, NULL);
    filp_close(tgt_file, NULL);
    return 0;
}

```

(3) 注册系统调用

vim arch/x86/entry/syscalls/syscall_64.tbl

468	common	schello	sys_schello
469	common	alcall	sys_alcall

(4) 编译内核

增量编译 make -j8

(5) 安装内核模块

sudo make modules_install

(6) 安装内核

sudo make install

(7) 重启系统

sudo reboot

4、编写用户态测试程序

cd ~

vim test_schello.c



```
LZH2311451@LZH2311451-pc: ~
文件(F) 编辑(E) 视图(V) 搜索(S) 终端(T) 帮助(H)
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define __NR_alcall 469

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("用法: %s <src_path> <dest_path>\n", argv[0]);
        return -1;
    }

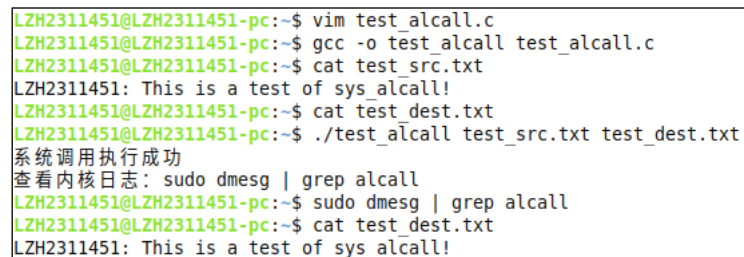
    long ret = syscall(__NR_alcall, argv[1], argv[2]);

    if (ret == 0) {
        printf("系统调用执行成功\n");
        printf("查看内核日志: sudo dmesg | grep alcall\n");
    } else {
        printf("系统调用执行失败, 返回值: %ld\n", ret);
    }

    return 0;
}
```

5、编译运行并验证结果

提前准备了一个 test_src.txt（有内容）和一个 test_dest.txt（无内容）用来测试。



```
LZH2311451@LZH2311451-pc:~$ vim test_alcall.c
LZH2311451@LZH2311451-pc:~$ gcc -o test_alcall test_alcall.c
LZH2311451@LZH2311451-pc:~$ cat test_src.txt
LZH2311451: This is a test of sys_alcall!
LZH2311451@LZH2311451-pc:~$ cat test_dest.txt
LZH2311451@LZH2311451-pc:~$ ./test_alcall test_src.txt test_dest.txt
系统调用执行成功
查看内核日志: sudo dmesg | grep alcall
LZH2311451@LZH2311451-pc:~$ sudo dmesg | grep alcall
LZH2311451@LZH2311451-pc:~$ cat test_dest.txt
LZH2311451: This is a test of sys_alcall!
```

这次实验的内核日志是没有输出的，在执行完程序后可以看到源文件中的内容被正确的复制到了目标文件中。

三、结论

本次实验成功实现了 alcall 自定义系统调用，完成了文件复制功能。这是第三次内核调用实验，又一次体验了修改内核源码、注册系统调用、编译安装内核及编写测试程序的流程，基本掌握了 Linux 系统调用的开发流程与底层工作机制，深入理解了内核态与用户态的数据交互逻辑。测试结果表明，alcall 系统调用可正确实现文件内容复制，达到了实验预期目标。